



ClickHouse

No GIL, Real Gains

Porting a C++ Python extension to free-threaded Python 3.14.

Auxten Wang · Technical Director @ ClickHouse · Creator of chDB

PyData Yerevan · 24 Jul 2026

~~GIL~~

↓

3.14t

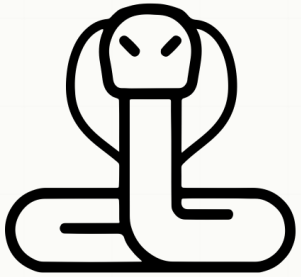
16 cores, one interpreter, zero subprocesses



Auxten Wang

@auxten · auxten.com

- **Technical Director @ ClickHouse**
- **Creator of chDB — acquired by ClickHouse in 2024**
- **20+ years of C/C++ · systems & data infrastructure**
- **Ex-Shopee · CovenantSQL · Baidu · 360 · 4Paradigm**



ClickHouse

Killing the GIL: 30 years of attempts

1996

Free-threading patch

Greg Stein removed the GIL from Python 1.4 with fine-grained locks.

✕ ~2× slower single-threaded — shelved. (`PyThreadState` survived!)

2007

python-safethread

Adam Olsen: objects owned per-thread, restricted sharing between threads.

✕ Too restrictive — faded away.

2016

Gilectomy

Larry Hastings, fine-grained locking again. Refcounts thrashed CPU caches.

✕ Single-thread perf collapsed — abandoned.

2021

nogil fork

Sam Gross: biased refcounting, immortal objects, mimalloc.

✓ Single-thread cost finally small → became PEP 703.

Meanwhile we coped with: multiprocessing (pickle tax) · asyncio (no parallelism) · subinterpreters

Guido's bar (2007): a GIL removal is welcome only if single-threaded performance does not decrease. Every attempt died on that bar — until one didn't.

The one that landed: free-threaded Python

PEP 703 · 3.13

The GIL becomes optional. Experimental build, ~40% single-thread penalty.

PEP 779 · 3.14

Officially supported. Specializing interpreter enabled; penalty down to ~5–10%.

Opt-in build

A separate binary: `python3.14t`. Wheels get a `t` suffix: `cp314t`.

```
$ python3.14t -c "import sys; print(sys._is_gil_enabled())"
False        # threads now run Python bytecode in parallel
```

Phase III — making it *the default* Python — is on the roadmap. This train is leaving.

Why it matters: Python finally spans the cores

Single-core speed plateaued years ago — machines got *wider* instead. Free-threading lets one Python process use the width.

Agent builders

Parallel tool calls & subagents in one process — threads share models, caches, vector indexes. No pickle, no IPC.

Desktop apps

Responsive UI thread + truly parallel background work. No `multiprocessing` spawn/pickle/packaging pain.

Servers & data tools

Thread-per-request finally uses all cores. One copy of the model — not 16 workers × 16 copies.

Same language, same process, 16× the compute. The rest of this talk: what that costs extension authors.

Extensions get nothing for free

- ✕ **One bad import poisons the well. Any extension without the opt-in flag → CPython silently *turns the GIL back on* for everyone.**
- ✕ **"Thread-safe because GIL" is now a data race. Every global, cache and lazy init loses its silent protection.**
- ✕ **Your abi3 wheel won't load. No stable ABI on 3.14t — new wheels, new CI.**

```
$ python3.14t -c "import _chdb"
RuntimeWarning: The global interpreter lock (GIL) has been enabled to load
module '_chdb', which has not declared that it can run safely without the GIL.
```

Free-threading is opt-in, per extension module. This talk: what opting in actually took.

Our patient — chDB: a rocket engine on a bicycle

The ClickHouse engine, embedded in your Python process — SQLite-simple, built to scan millions of rows.

```
# pip install chdb
import chdb
chdb.query("SELECT count() FROM file('hits.parquet')")
```

- **In-process:** no server, no ports — just a module.
- **Same engine:** ClickHouse SQL, JOINs, vectors.
- **Multi-threaded C++ inside** — remember this bit.



Local like SQLite. Analytical like ClickHouse.

It drops into your Python data stack

70+ formats

Parquet, CSV, JSON, Arrow, ORC — read & write, no converters.

Zero-copy Pandas / Arrow

Query a live DataFrame in place: `SELECT … FROM Python(df)`

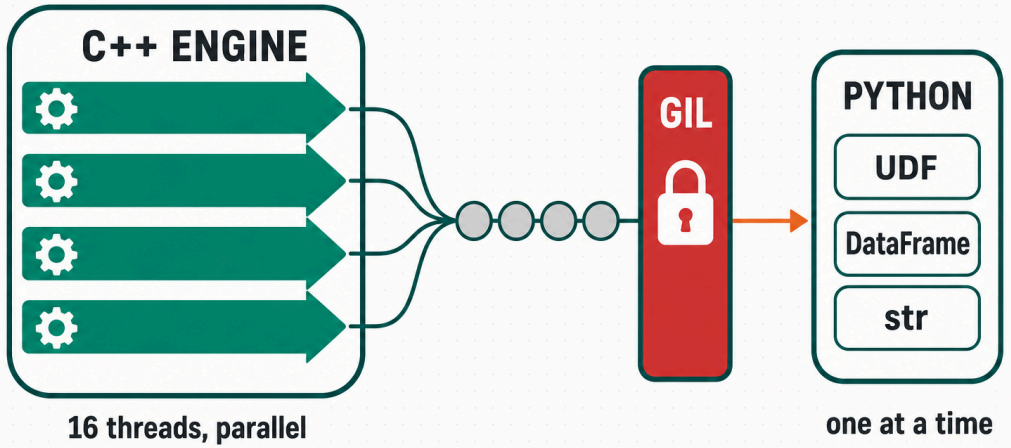
Python UDFs

Your Python functions callable from SQL — keep this one in mind too.

No ClickHouse knowledge needed today. For this talk, chDB is just "a multi-threaded C++ engine behind pybind11, reading Python objects."

Parallel engine × serial glue = serial query

The C++ engine releases the GIL and scales fine — but every stream queues on it to touch anything Python.



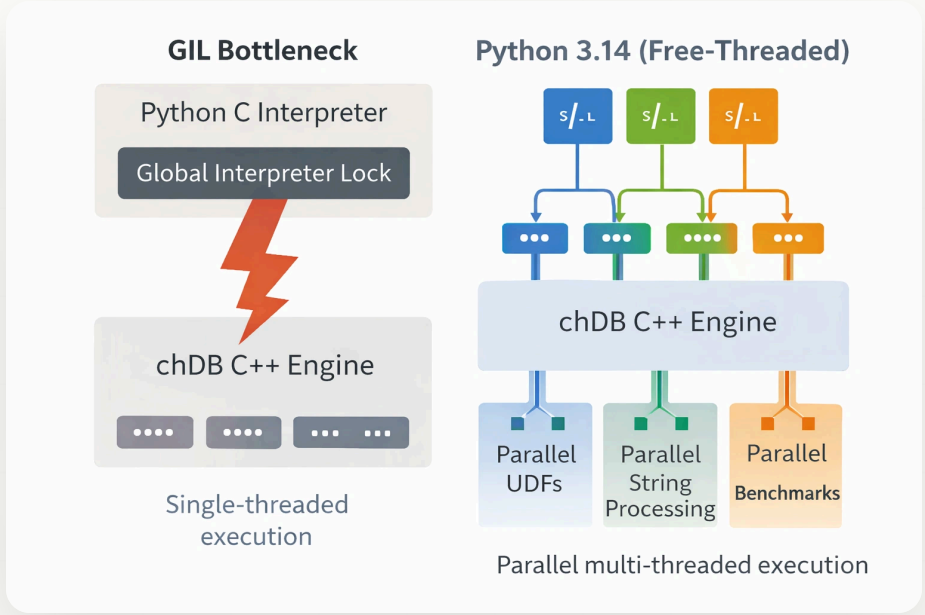
The GIL didn't slow our engine down. It serialized every conversation with Python.

Three changes that made no-GIL pay off

01 **Move Python UDF execution in-process**
Kill the subprocess pipeline first.

02 **Declare the module GIL-independent**
`Py_mod_gil_not_used` — then survive the audit it implies.

03 **Drop to low-level CPython string access**
On hot paths, the friendly Unicode APIs had to go.



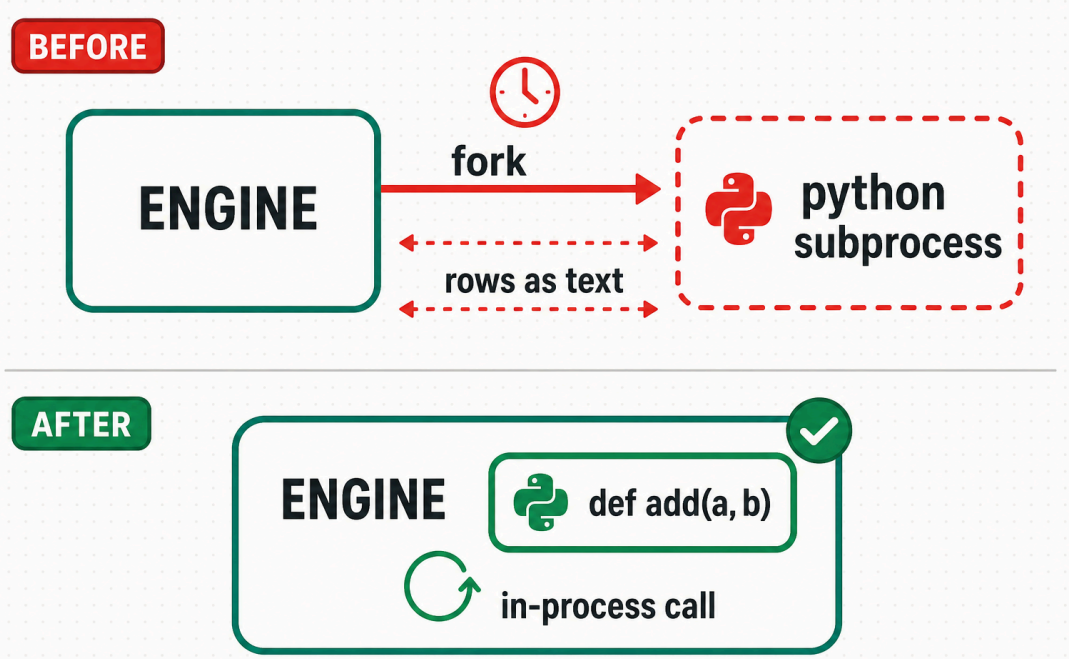
Plus the honest part: **what broke**, what didn't scale, and where the stable ABI stopped being an option.

UDFs ran out of process

The old decorator wrote a script + XML config; the engine forked Python and piped rows through it as text.

- ✕ Fork + interpreter startup on the query path
- ✕ Rows serialized as TabSeparated text over stdin/stdout
- ✕ Every argument arrives as `str` — no types
- ✕ No shared state with your process; imports re-run

The GIL wasn't even the bottleneck yet — the `fork()` was.



After: native in-process UDFs

C++ holds your `py::function` and calls it from the engine pipeline. Typed args, zero subprocesses.

```python
import chdb
from chdb.sqltypes import INT64

@chdb.func([INT64, INT64], INT64)  # in-process
def add(a, b):
    return a + b

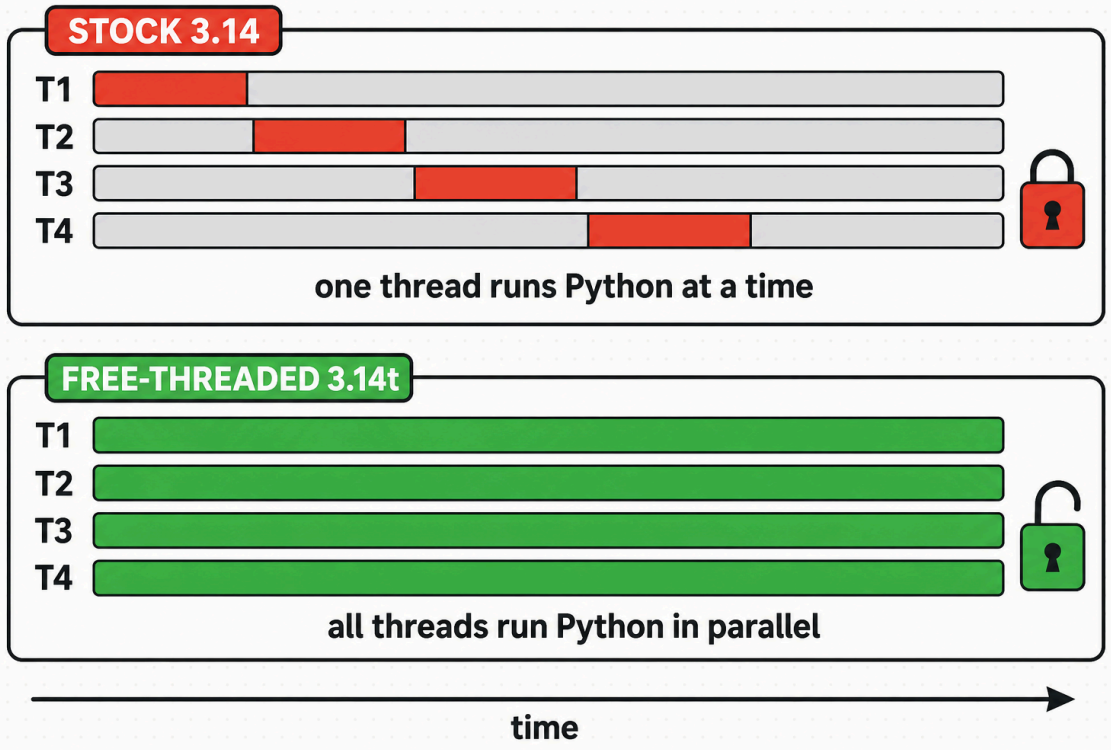
chdb.query("SELECT add(x, y) FROM t")
```

```cpp
// C++: PythonScalarUDF::executeImpl
// (called from engine worker threads)
py::gil_scoped_acquire acquire;   // <-- THE lock

for (size_t row = 0; row < rows; ++row)
{
    py::tuple py_args = box_arguments(row);
    py::object out    = func(*py_args); // your fn
    insert_into_column(result, out);
}
```

Faster already — but see that `gil_scoped_acquire`? On a stock build, every engine thread queues on it.

This is what no-GIL was made for



- **Stock 3.14: 16 streams hit the UDF → 16 threads take turns on one GIL. Effectively single-core.**
- **3.14t: the same `acquire` becomes per-thread bookkeeping — streams run your function in parallel.**
- **Your UDF must be thread-safe too — it now runs concurrently.**
- **Per-row boxing still costs: no-GIL removes the contention, not the call overhead.**

Order matters: go in-process *first*, then remove the GIL. A subprocess pipeline gains nothing from free-threading.

Declare the module GIL-independent

One line — and a promise you now have to keep. Without it, importing your module quietly re-enables the GIL for everyone.

- **pybind11 ≥ 2.13:** `py::mod_gil_not_used()`
- **Raw C API:** `Py_mod_gil` **slot (multi-phase init)**
- **Legacy single-phase:** `PyUnstable_Module_SetGIL()`

```
// chDB: gate it behind the build flag
#ifdef CHDB_FREE_THREADING
PYBIND11_MODULE(_chdb, m, py::mod_gil_not_used())
#else
PYBIND11_MODULE(_chdb, m)
#endif

// raw C API equivalent (multi-phase init):
static PyModuleDef_Slot slots[] = {
    {Py_mod_gil, Py_MOD_GIL_NOT_USED},
    {0, NULL},
};
```

This line is the *easy* part. It's a claim about every global in your codebase.

The global-state audit: what we actually found

SHARED STATE	RACE UNDER FREE-THREADING	FIX
Python import cache	lazy `import` / `getattr` from many threads	`std::call_once` + mutex
UDF registry	register vs. lookup during queries	`std::shared_mutex`
Engine singleton	two threads boot the engine at once	mutex-guarded init
Connection state	concurrent connect / close / query	per-client mutex
Per-row cache lookups	hot-path contention once GIL is gone	hoist out of the row loop

Textbook C++ concurrency — the work is finding it. Our import cache's "safe" lock-free fast path had been a data race for years. The GIL was its alibi.

The nastiest bug: lazy init on a GIL-free thread

Trigger

First `NULL` in a string column → scan thread lazily resolves `pandas.NaT`... with no Python thread state.

Crash

Segfault inside `_PyObject_Malloc` on CPython 3.12+ — deep in the allocator, nowhere near the actual bug.

Hang

The fatal-signal handler then deadlocks. No traceback, no exit. The query just hangs, forever.

```
/// The fix — in prepareColumnCache(), while still holding the GIL:
/// pre-resolve everything GIL-free scan threads will compare against.
import_cache.pandas.NaT();
import_cache.pandas.NA();
```

Audit every lazy initialization reachable from a non-Python thread. Resolve it eagerly, under the GIL.

Strings are the hot path

Analytics = millions of strings crossing the C++/Python border per query. Our regular wheel compiles with `Py_LIMITED_API` (one abi3 wheel, cp39+) — which hides the fast Unicode internals.

- ✕ **Limited API: no `PyUnicode_DATA`, no `KIND`, macros become out-of-line calls**
- ✕ **Our workaround: a per-version *shim library* — every hot string access pays a cross-`.so` function call**

```
// Stable-ABI build: hot loop calls into a shim .so
// (libpybind11nonlimitedapi_chdb_3.X.so)
pybind11::non_limited_api::getPyUnicodeUtf8(
    obj, data, length, kind, ...);
// one non-inlinable call per string value
```

Fine for control flow. `O(rows)` on the scan path? That's a tax on every value.

3.14t: go straight to the bytes

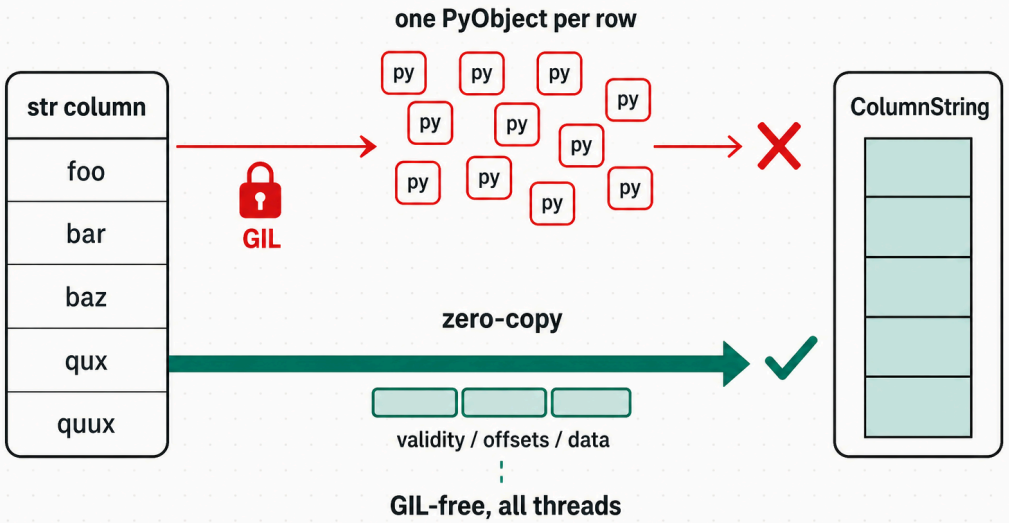
The free-threaded build can't use the Limited API anyway — so it drops to direct CPython Unicode access:

```
#ifdef CHDB_FREE_THREADING                          // full C API available
if (PyUnicode_IS_COMPACT_ASCII(obj)) {
    data = (const char *) PyUnicode_1BYTE_DATA(obj);  // direct pointer, no copy
    len  = PyUnicode_GET_LENGTH(obj);
}
else if (unicode->utf8)                             // CPython's cached UTF-8
    { data = unicode->utf8;  len = unicode->utf8_length; }
else if (PyUnicode_IS_COMPACT(obj))
    kind = PyUnicode_KIND(obj);                     // 1/2/4-byte units -> utf8proc
else {
    py::gil_scoped_acquire acquire;                 // rare legacy strings only
    data = PyUnicode_AsUTF8AndSize(obj, &len);
}
#endif
```

ASCII (most analytical data) = a pointer read. No shim, no allocation, safe from any thread on 3.14t.

The biggest win: skip PyObject entirely

pandas 3.x `str` columns are **Arrow-backed**: plain C buffers, scanned directly on all streams — *no GIL at all*.



object dtype (before) — 219.2 s

Arrow zero-copy (after) — 20.1 s

ClickBench, 50M-row hits DataFrame, all queries, pandas 3.x, hot run · **10.9×** total · geomean **4.5×** · string-heavy up to **64×**

Priority order on hot paths: no PyObject > cheap PyObject access > friendly PyObject APIs.

What broke, what surprised us

- ✗ **The silent GIL re-enable. One undeclared dependency → your "no-GIL benchmark" measures the GIL. Check `sys._is_gil_enabled()` *inside*.**
- ✗ **Our first GIL-free object scan was wrong. Borrowed refs + lazy imports — we re-acquired the GIL. Durable wins: Arrow buffers, true FT builds.**
- ✗ **Crashes hide as hangs. Allocator segfault → signal-handler deadlock. Budget time for debugging *silence*.**
- ✗ **The stable ABI stopped being an option. No abi3 on 3.14t → `cp313t`/`cp314t` wheels, vendored pybind11, doubled CI.**
- ✗ **No-GIL didn't speed up what was already parallel. Only the Python touchpoints improved — and single-threaded got ~5–10% slower.**

Free-threading didn't *create* these bugs. It revoked the amnesty they'd been living under.

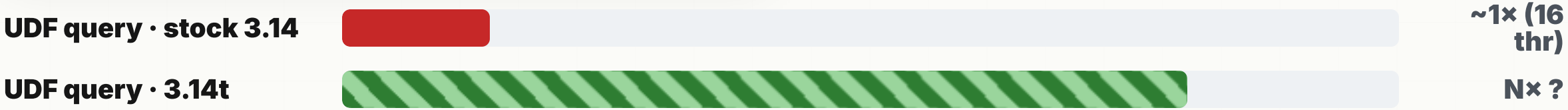
Numbers from a 16-core machine DRAFT

Shipped & measured

- ✓ ClickBench, 50M rows: 219.2s → 20.1s (10.9×)
- ✓ String-heavy queries: up to 64×
- ✓ CI guard: 5M-string ingestion microbench on 3.14

To re-run for this talk TODO

- UDF-heavy query: threads 1→16, stock vs 3.14t
- Object-dtype string scan: stock vs 3.14t
- Single-thread regression: 3.14 vs 3.14t



Striped bar = placeholder — final scaling curve to be measured on the 16-core box before the talk.

When does no-GIL actually help?

It pays when…

- ✓ Many threads must execute Python (UDFs, callbacks)
- ✓ Many threads must read PyObjects (object-dtype columns)
- ✓ The GIL was your *measured* bottleneck, not a hunch

It doesn't when…

- ✗ Your C++ already releases the GIL for the heavy work
- ✗ You can avoid Python objects entirely (Arrow, buffers, NumPy)
- ✗ The workload is single-threaded — you pay ~5–10% for nothing

Best result in our port: zero-copy beat lock-removal. Free-threading fixes what zero-copy can't reach — running actual Python in parallel.

The porting checklist

1. Run tests on `3.14t` as-is — watch for the GIL re-enable `RuntimeWarning` (yours *and your deps'*).
2. Kill architectural serialization first (subprocesses, single workers) — give threads something to win.
3. Inventory global state: caches, registries, singletons → `call_once` / mutex / `shared_mutex`.
4. Hunt races the GIL was masking: unsynchronized fast-path reads, borrowed refs shared across threads.
5. Pre-resolve lazy imports/attributes reachable from non-Python threads — eagerly, under the GIL.
6. Plan wheels: no Limited API on FT builds → version-specific `cp313t`/`cp314t` lanes in CI.
7. Re-do hot paths with the full C API behind `#ifdef Py_GIL_DISABLED` (strings first).
8. Only then declare `Py_mod_gil_not_used` — and benchmark scaling *and* the single-thread regression.

Three takeaways

01 **No-GIL is opt-in, and the opt-in is an audit.**
The declaration is one line; the promise covers every global you own.

02 **Restructure before you parallelize.**
In-process beats out-of-process; zero-copy beats lock-free; then free-threading multiplies what's left.

03 **The GIL was an amnesty, not a feature.**
3.14t revokes it. The bugs it exposes were already yours.

Try it on your own extension

```
pip install chdb           # try the engine
python3.14t your_tests.py # try the future
```

Code & slides

`github.com/chdb-io/chdb`
`auxten.com · @auxten`

Read more

py-free-threading.github.io
PEP 703 · PEP 779 · chDB docs:
clickhouse.com/docs/chdb

Python can finally scale across cores.
Now it's our extensions' turn.





Q&A

`github.com/chdb-io/chdb` · `auxten.com`

No GIL, real gains — thank you, Yerevan!